CompSci 330: Design and Analysis of Algorithms
Summer 2024, Homework 4
Due: Thu 8/1 @ 11:59pm

Directions

How to Do Homework. We recommend the following three step process for homework to help you learn and prepare for exams.

1. Give yourself 15-20 minutes per problem to try to solve on your own, without help or external materials, as if you were taking an exam. Try to brainstorm and sketch the algorithm for applied problems. Don't try to type anything yet.

2. After a break, review your answers. Lookup resources or get help (from peers, office hours, Ed discussion, etc.) about problems you weren't sure about.

3. Rework the problems, fill in the details, and typeset your final solutions.

Submission. Your written solutions should be typeset and submitted as a single PDF on Gradescope. LaTeX[1] is preferred but not required. If you use another editor for your solutions (such as Microsoft Word), you should convert the final document to a PDF, confirm that the symbolic math from the equation editor is properly formatted, and submit the PDF. You must mark the locations of your solutions to individual problems on Gradescope as explained in the documentation. Implementation problems will request that you submit code separately on Gradescope to be autograded.

Writing Expectations. If you are asked to provide an algorithm, you should clearly and unambiguously define every step of the procedure as a combination of precise sentences in plain English or pseudocode. If you are asked to explain your algorithm, its runtime complexity, or argue for its correctness, your written answers should be clear, concise, and should show your work. Do not skip details but do not write paragraphs where a sentence suffices. Results covered in lecture may be referenced and used directly without any justification of their correctness (unless asked for).

Collaboration. You can (and should!) work with a partner, in which case you will submit a single solution as a group on Gradescope ***by adding your partner to your submission on Gradescope*** (see here for directions). ***Always cite anyone/anything consulted*** outside of course materials, including a brief description of their contribution to your work. See also our policy on collaboration on the course webpage.

Grading. Theory problems will be graded on an S/U scale (for each sub-problem). Applied problems typically have a separate autograder where you can see your score. **The lowest scoring problem is dropped.** See the course assignments webpage for more grading details.

[1] If you are new to LaTeX, you can download it for free at latex-project.org or you can use the popular and free (for a personal account) cloud-editor overleaf.com. We also recommend overleaf.com/learn for tutorials and reference.

Problem 1

The town of Thirtyville is preparing for a large expansion. Naturally, the mayor of Thirtyville needs to decide how the electrical network for the town should be designed. Let $G = (V, E)$ be a connected undirected graph with positive edge weights $w : E \to \mathbb{R}^+$ representing the expanded town. The vertices in V represent locations and the edges in E represent potential power lines that can be built, where the weight $w(e)$ of an edge $e \in E$ is the cost in dollars to build that power line.

The set of vertices V is composed of two disjoint sets $H \subset V$ and $P \subset V$; $V = H \cup P$. The vertices in H represent houses, and the vertices in P represent where power plants will be built. Describe an algorithm to compute the cheapest subset of power lines $L \subseteq E$ to build so that every house $h \in H$ is reachable from a power plant $p \in P$ via the power lines in L. Note that houses do not need to have a direct power line to a power plant; a path composed of power lines suffices. Justify the correctness of your algorithm. A formal proof is not necessary. Analyze the runtime. *[Hint: Reduce the problem to computing a minimum spanning tree in a different graph. Include all parts of a reduction in your answer.]*

Problem 2

While you were solving the previous problem, the mayor has realized building power plants at the previously considered locations will be far too costly. Instead, they must solve the following problem (with the same setup as before, for completeness): Let $G = (V, E)$ be a connected undirected graph with positive edge weights $w : E \to \mathbb{R}^+$ representing the expanded town. The vertices in V represent locations and the edges in E represent potential power lines that can be built, where the weight $w(e)$ of an edge $e \in E$ is the cost in dollars to build that power line.

Now, a power plant can be built at ***any location in V*** for a ***cost of z dollars***, where $z > 0$ is a given parameter (in this problem, there is no partition of V into different kinds of vertices). The goal is now to select locations $P \subseteq V$ to build power plants ***and*** which power lines $L \subseteq E$ to build so that ***every*** location in V is reachable from a power plant $p \in P$ via the power lines in L. Furthermore, the total cost of the power plants and power lines must be minimized.

For example, a feasible solution with cost $z|V|$ is to set $P = V$ and $L = \varnothing$, meaning that a power plant is built at every location, which is valid because every location is trivially reachable from itself with a power plant. For another example, a feasible solution is to build a single power plant (somewhere) and decide which edges to connect all locations it. In between these extremes is the possibility to build more than one but less than $|V|$ power plants.

Describe an algorithm for this problem. Justify the correctness of your algorithm. A formal proof is not necessary. Analyze the runtime. *[Hint: Reduce the problem to computing a minimum spanning tree in a different graph (perhaps similar, but not the same, as that for the previous problem). Include all parts of a reduction in your answer.]*

Problem 3

Phoebe is an artisan potter that specializes in terracotta. She offers a vast catalog of vases with various sizes, colors, and shapes, and she has just received an immensely large and complicated order for n different designs of vases, $V_1, V_2, \ldots, V_n$. Each design V_i takes a distinct number of minutes, s_i, for Phoebe to first sculpt by hand, and then another distinct number of minutes, f_i, for her to finish by firing it in her kiln. For purposes of this problem, suppose her kiln can fit any number of sculpted vases to fire and that it is OK for fully-fired vases to remain in the kiln until all vases are done firing.

Phoebe can only sculpt one design at a time, but immediately after she finishes sculpting a vase she adds it to the others in the kiln (if any) so that she can begin sculpting another vase. Once all vases are sculpted and fired in the kiln for at least as long as they all require, she pulls them all out at once and her job is done. Your goal is to help Phoebe determine which order she should sculpt the n designs in order to minimize the time when her job is done.

(a) Prove that ordering the designs in increasing order of s_i is not optimal by providing a counterexample by providing an input for which the ordering does not yield the best possible end time.

(b) Same as the last part, but with ordering in increasing order of $s_i + f_i$.

(c) State an ordering for Phoebe to use and prove that it is optimal. You may find the following proof template useful[2]:

(i) Compare your ordering to that of an arbitrary optimal ordering: If the orderings are not identical, then there are some pairs of designs in the optimal ordering that are out-of-order with respect to your ordering.

(ii) Consider the optimal ordering ***with the fewest such pairs***.

(iii) Show that swapping two adjacent out-of-order designs in that ordering reduces the number of out-of-order pairs without increasing the overall end time.

(iv) Conclude that, because the resulting order has fewer out-of-order pairs and optimal end time, your ordering is indeed optimal by contradiction.

[2]This is a *proof by smallest counterexample* instead of induction, which is functionally equivalent but perhaps more intuitive here. See the notes on Induction for a more in-depth discussion about this proof technique.

Problem 4

Let X be a set of n intervals on the real line labeled $1, \ldots, n$. We say that a subset of intervals $Y \subseteq X$ *covers* X if the union of all intervals in Y is equal to the union of all intervals in X. The *size* of a cover is just the number of intervals.

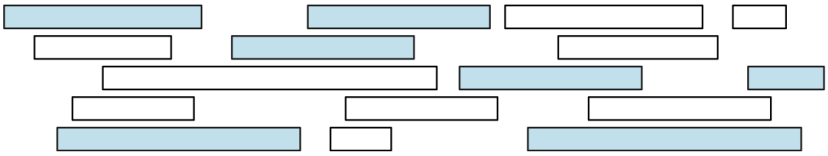

Figure 1. A set of intervals, with a cover (shaded) of size 7 (not necessarily the optimal cover).

Describe a greedy $O(n \log n)$-time algorithm to compute the smallest cover of X. Assume that your input consists of two arrays $L[1..n]$ and $R[1..n]$, where $(L[i], R[i])$ is the i-th interval in X. Note that X as given is ordered arbitrarily. For simplicity, assume that no two endpoints of intervals are equal; that is, all elements of L, R are distinct. Prove the correctness of the algorithm (including an exchange argument) and analyze its runtime complexity.